

Sistemas embebidos

Herramientas de software para la generación de código μ C ATmega328P

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2025-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

Preguntas...

¿Sabes que es la **lenguaje ensamblador**?
¿Que es un **mnemonico**?
¿Qué es una **directiva**?



Programación del ATmega328P

Existen diversas **herramientas de desarrollo** dependiendo el objetivo del proyecto

Programación en lenguajes de alto nivel

Permiten desarrollar programas utilizando **lenguajes de alto nivel** (generalmente C o C++) **acortando** el tiempo de desarrollo. Las principales opciones son:

- Plataforma de **Arduino** (híbrido de C/C++)
- **AVR GCC Toolchain** (lenguaje C/C++)

Programación en lenguajes de bajo nivel

Permite utilizar el **conjunto de instrucciones** del microcontrolador para realizar programas con **acceso a bajo nivel** de **todos los recursos** del microcontrolador.

Programación en lenguajes de alto nivel

Plataforma de Arduino

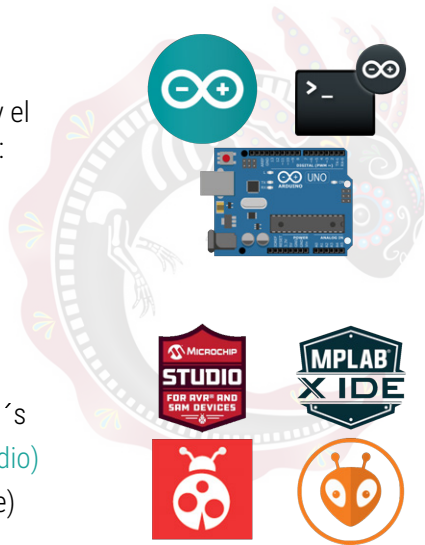
Incluye el **hardware** (tarjeta de desarrollo) y el **software** (IDE) con las siguientes opciones:

- **Arduino IDE 1.8.19** (IDE clásico)
- **Arduino IDE 2.x** (IDE y CLI)

AVR GCC Toolchain

Compilador de C/C++, **ensamblador** y **enlazador**. Es la base de los siguientes IDE´s

- **Microchip Studio** (Integra ATMELE Studio)
- **MPLabIDE X** (IDE actual del fabricante)
- **PlatformIO** (se integra con VSCode)



Programación en lenguaje ensamblador

AVR GCC Toolchain 8 bits

Toolchain basado en GCC que contiene el **compilador** de C/C++, el **ensamblador** y **enlazador** en **línea de comandos**. Se pueden integrar con **cualquier IDE** y esta disponibles en **windows, GNU/Linux y OSX**.

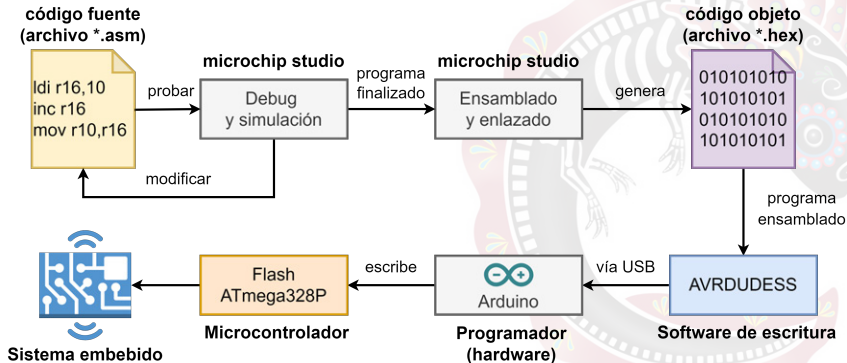
Microchip Studio

ATMEL Studio se integró a **Microchip studio** el cual fue el IDE del fabricante para los productos de microchip. **Descontinuado** para nuevos productos y sólo disponible en **windows**.

MPLabIDE X

El **IDE actual** recomendado por el fabricante. Disponible en **windows, GNU/Linux y OSX**.

Flujo de trabajo



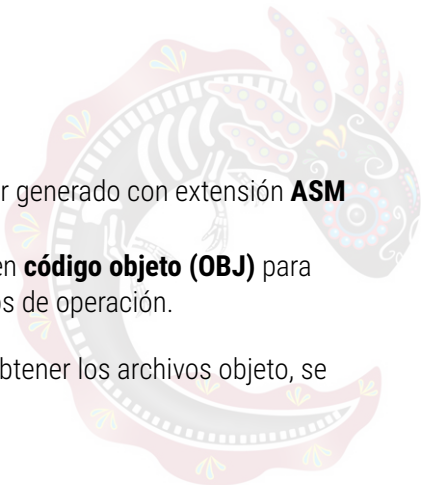
Archivos de Programación

Extensión de archivo

Todo programa en **ensamblador** debe ser generado con extensión **ASM**

Al ser ensamblado se generan archivos en **código objeto (OBJ)** para reconocer los **mnemónicos** como códigos de operación.

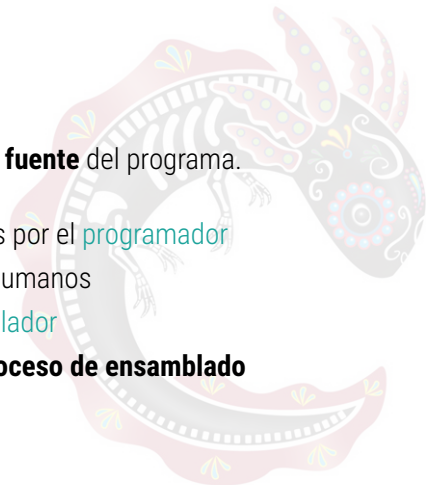
Dependiendo del programa usado para obtener los archivos objeto, se generan **tres tipos de archivos**:



Archivo ASM

Es el **documento** que contiene el **código fuente** del programa.

- Contiene las **instrucciones** escritas por el **programador**
- Utiliza **mnemónicos** legibles para humanos
- Debe seguir la **sintaxis** del **ensamblador**
- Es el **archivo de entrada** para el **proceso de ensamblado**



Archivo LSS

Es un **documento auxiliar** de listado que contiene **información detallada** del **proceso de ensamblado**.

Información que contiene:

- **Dirección de memoria** ocupada
- **Códigos de operación**
- Listado del **código fuente**
- **Asignación de valores** de **variables y constantes** definidas por el programador



Importante

En este archivo es posible **detectar errores** al ensamblar.

Archivo HEX

Es el archivo donde residen los **códigos de operación** del programa ya **ensamblado**. Este archivo es el que se escribe en **memoria flash**.

Formato del archivo HEX:

```
1 : nn aaaa 00 dd dd dd dd dd ... dd (ERC);  
2 :  
3 : 00 0000 01 FF; linea final
```

- **nn:** Número de bytes de datos
- **aaaa:** Dirección de memoria
- **dd:** Datos (códigos de operación)
- **ERC:** Código de verificación

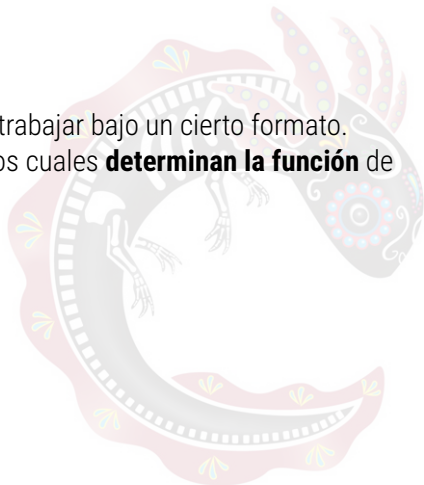
Formato del Programa Fuente

Estructura

Para editar un programa fuente se debe trabajar bajo un cierto formato. Este formato está dividido en **campos**, los cuales **determinan la función** de los distintos **elementos** en un programa.

Existen cuatro campos básicos:

- 1 Campo de **etiquetas**
- 2 Campo de **operaciones**
- 3 Campo de **operandos**
- 4 Campo de **comentarios**



Campo de Etiquetas

Este campo contiene las **etiquetas** asignadas a cualquier **línea de programa**.

Reglas para las etiquetas:

- Deben empezar en la **primera columna**
- Aceptan caracteres **alfanuméricos**
- Sus **limitadores** son los caracteres de dos puntos ":", o el **tabulador vertical**

Ejemplos:

```
1 inicio:      LDI R16, 10
2              INC R16
3 fin:         RJMP inicio
```

Campo de Operaciones

Este campo contiene los **mnemónicos** de las **instrucciones** así como algunas **directivas** o **pseudo instrucciones** del ensamblador.

Características:

- Las directivas o pseudo instrucciones **indican** cómo **procesará** el ensamblador un **operando** o una **sección** del programa
- Algunas directivas **generan y almacenan** **información** en la memoria y otras no

Principales directivas:

- CSEG, DSEG, ESEG, DB, DW, ORG, EQU, BYTE, DEF

Directiva CSEG

Marca el inicio de un **segmento de código** y está orientada al uso de la **memoria Flash**. Es el **segmento por omisión**, no es necesario especificarla cuando un programa no va a utilizar **otros segmentos**.

Sintaxis:

[etiqueta] .CSEG

Ejemplos:

```
1      .CSEG                ; Inicia un segmento de codigo
2      LDI R16, 0x25        ; R16 = 0x25
3      STS var1, R16        ; escribe en un espacio de SRAM
```

Directiva DSEG

Señala el inicio de un **segmento para los datos** en la **SRAM**, es útil para **reservar** algunas localidades para un propósito especial.

Sintaxis:

[etiqueta] .DSEG

Ejemplos:

```
1      .DSEG
2  var1:  .BYTE 1      ; Variable de 1 byte
3  var2:  .BYTE 10     ; Variable de 10 bytes
```

Directiva ESEG

Indica que la siguiente **información** será guardada en memoria **EEPROM**, memoria destinada a **variables** que se deben **conservar** en ausencia de **energía**. Al ensamblar un programa con un segmento de EEPROM, **se crea un archivo** con extensión **.eep**; este debe ser descargado al **MCU**.

Sintaxis:

[etiqueta] .ESEG

Ejemplos:

```
1      .ESEG                ; Inicia un segmento de EEPROM
2  eevar1:  .DB  0x3F        ; Variable en memoria EEPROM
3  eevar2   .DW  0xFFFF      ; Variable en memoria EEPROM
```


Directiva DB y DW

Sirven para definir **constantes o variables** en los segmentos de **código** y de **EEPROM**. **DB** es para datos que ocupan **1 byte** (*Define Byte*) y **DW** para datos de **2 bytes (16 bits)** (*Define Word*). Si se requiere **más de un dato**, deben organizarse en **una lista** con los elementos separados por comas.

Sintaxis:

```
[etiqueta] .DB/.DW [VALOR1], [VALOR2], ..., [VALORn]
```

Ejemplos:

```
1      .CSEG                ; Constantes en memoria Flash
2  cons1:    .DB  0x33
3  consts1:  .DB  0, 255, 0b01010101, 128, 0xAA
4
5      .ESEG                ; Constantes o variables en EEPROM
6  eevar1:   .DB  0x37
7  eelist2:  .DW  0, 0xFFFF, 10
```

Directiva EQU

La directiva **EQU** se utiliza para realizar **definiciones** que posteriormente se pueden usar en las **instrucciones**.

Sintaxis:

```
[etiqueta] .EQU [alias] = [VALOR]
```

Ejemplos:

```
1 .EQU PortA = 0x25
2
3 .CSEG          ; Inicia el segmento de código
4 CLR R2         ; Limpia el registro 2
5 OUT PortA, R2  ; Escribe en PortA
```

Directiva ORG

Se utiliza para ubicar la **información** subsecuente en cualquiera de los **segmentos de memoria**. En la **SRAM** o **EEPROM** indica la **dirección** a partir de la cual se colocan las **variables**, y en la **Flash** puede ubicar **constantes** o **código**.

Sintaxis:

```
[etiqueta] .ORG [direccion]
```

Ejemplos:

```
1      .DSEG
2      .ORG 0x120      ; Dir 0x120 del segmento de datos
3  var1: .BYTE 1      ; Variable
4
5      .CSEG
6      .ORG 0x010
7  inicio: MOV R1, R2  ; Código ubicado en la dir 0x10
```

Funciones HIGH y LOW

Las funciones **HIGH** y **LOW** se utilizan para separar constantes de 16 bits; con **HIGH** se obtiene la **parte alta** y con **LOW** la **parte baja**.

Sintaxis:

HIGH (VALOR)
LOW (VALOR)

Ejemplos:

```
1      LDI R27, HIGH (578) ; El valor de la constante  
2      LDI R26, LOW (578) ; se identifica claramente
```

Directiva BYTE

La directiva **BYTE** sirve para **reservar uno o más bytes** en **SRAM** o **EEPROM** para el **manejo de variables**, la cantidad de bytes debe **especificarse** y **el espacio no es inicializado**, solo queda **reservado**.

Sintaxis:

```
[etiqueta] .BYTE [NumeroDeBytes]
```

Ejemplos:

```
1      .DSEG
2  var1:  .BYTE  1           ; Variable de 1 byte
3  var2:  .BYTE 10           ; Variable de 10 bytes
4
5      .CSEG
6  STS var1, R17              ; Acceso directo a var1
7  LDI R30, LOW(var2)         ; Z apunta a low de var2
8  LDI R31, HIGH(var2)        ; Z apunta a high de var2
9  ST Z, R1                   ; Acceso a var2 mediante Z
```

Directiva DEF

La directiva **DEF** establece un **nombre simbólico** a un **registro**, para dar **claridad** a los programas. El nombre puede ser empleado en **cualquier parte** de un programa y un registro puede tener **más de un nombre**.

Sintaxis:

```
[etiqueta] .DEF [alias] = [registro]
```

Ejemplos:

```
1 .DEF limite = R16          ; define registro limite
2 .DEF conta  = R20          ; define registro contador
3
4     LDI limite, 10          ; cargar valor maximo en limite
5     CLR conta               ; inicializar contador en cero
6
7 loop:  CP  conta, limite     ; comparar contador con limite
8        BRSH fin              ; saltar a fin si conta >= limite
9        INC conta             ; incrementar contador
```

Campo de Operandos

En este campo se tienen los **elementos** sobre los cuales se **ejecutarán** las operaciones.

Reglas:

- Cuando se tienen **más de dos operandos**, estos se separan por una **coma ","**
- Sus **limitadores** son los caracteres de punto y coma ";", o el **tabulador vertical**

Ejemplos:

1	<code>INC R16</code>	<code>; Un operando</code>
2	<code>MOV R12, R10</code>	<code>; Dos operandos</code>
3	<code>SBI PINB, 5</code>	<code>; Dos operandos</code>
4	<code>SJMP inicio</code>	<code>; Un operando</code>

Campo de Comentarios

Este campo sirve para **comentar información** que puede ser útil para **explicar al programador** funcionalidades básicas del código.

Característica:

- El ensamblador **detecta un comentario** cuando este se precede por un punto y coma ";"

Ejemplos:

```
1 ; Este es un comentario completo en una linea
2     LDI R16, 0xFF      ; Cargar 255 en el registro 16
3     OUT PORTB, R16    ; Enviar valor al Puerto B
4 loop:                    ; Etiqueta para el bucle
5     SJMP loop          ; Bucle infinito
```


Ejemplo de Programa Completo

```
1 ; Programa de ejemplo ATmega328P
2 ; Author : Rodrigo Vazquez
3
4 .dseg
5 data: .byte 1
6 tabla: .byte 10
7
8 .cseg
9 org 0x00
10
11 start:
12     ldi r16, 0xff    ; r16 <- 255
13     out ddrb, r16    ; configura puerto B como salida
14     clr r16          ; r16 <- 0
15     out portb, r16   ; B <- 0
16
17 loop:
18     sbi pinb, 5      ; pone en 1 el pin 5 del puerto B
19     rjmp loop        ; salto a loop
```

Análisis del Ejemplo

Elementos identificados en el programa:

- **Campo de etiquetas:** start, loop, data, tabla
- **Campo de operaciones:** .org, .byte, ldi, out, clr, sbi, etc.
- **Campo de operandos:** 0x00, 0xff, r16, ddrb, etc.
- **Campo de comentarios:** Todas las líneas que inician con ";"

Nota

Observe cómo cada campo está claramente separado y cumple con las reglas establecidas.

Escribir el programa en el microcontrolador



Configuración del AVRDUDESS

- 1 **Programmer:** **arduino** como programador utilizando el **protocolo STK500 v1**.
- 2 **Port:** el **puerto USB** al cual esta conectada la tarjeta (**aparece como COM y un número**).
- 3 **Baud rate:** **velocidad** de transferencia, **siempre** debe ser **115200**.
- 4 **MCU:** **modelo** del microcontrolador, en este caso **ATmega328P**.
- 5 **Flash:** **ubicación** del archivo ***.hex**.

